

shogiAI 數據科學理論與應用報告

Policy-only 將棋模型、資料前處理與 Alpha-beta 搜尋整合

shogiAI 專案

2026 年 6 月

摘要

本報告說明 shogiAI 專案的數據科學流程與 AI 應用。專案以大量 CSA 將棋棋譜作為資料來源，從 MySQL 匯出「局面、下一手」訓練樣本，將局面編碼成 $43 \times 9 \times 9$ 張量，將走法編碼成 13,689 類 move label，訓練 policy-only 卷積神經網路。新版模型移除 value head，專注於預測實戰下一手與排序合法候選手；局面計算則交由 alpha-beta 搜尋、quiescence search、transposition table、killer move、history heuristic 與手工評估函數完成。最新遠端訓練使用 4,855,173 筆樣本，測試 Top-1 accuracy 為 41.10%，Top-5 accuracy 為 74.49%。

目录

1	數據科學目標	3
2	問題定義	3
3	資料來源與最新規模	3
4	資料前處理流程	4
5	局面特徵編碼	5
6	行棋方視角旋轉	5
7	走法標籤設計	5
7.1	一般移動	6
7.2	打入持駒	6
7.3	總類別數	6

目錄	2
8 從 Policy/Value 改成 Policy-only	6
8.1 Value 標籤訊號太粗	6
8.2 投降資料使局面價值不精準	6
8.3 小型 CNN 容量有限	7
8.4 實際應用以候選手排序為主	7
9 Policy-only 模型架構	7
10 訓練方法	7
10.1 Loss Function	7
10.2 Optimizer	8
10.3 Learning Rate Scheduler	8
10.4 Game-level Split	8
11 最新訓練結果	8
12 結果解讀	9
13 模型推論與快取	9
14 Alpha-beta 搜尋概論	9
14.1 Minimax 與 Negamax	10
14.2 Alpha-beta Pruning	10
15 shogiAI 搜尋技術	10
15.1 Quiescence Search	10
15.2 Transposition Table	10
15.3 Killer Move	10
15.4 History Heuristic	11
15.5 Late Move Reduction	11
16 手工評估函數	11
17 Policy Network 如何結合 Alpha-beta	11
18 Opening Book	12
19 模型互打與評估	12
20 前端展示與應用	13

目录	3
21 限制與改進方向	13
21.1 模型容量限制	13
21.2 資料標籤限制	13
21.3 Legal Move Mask	13
21.4 自我對弈	14
21.5 Value 的未來可能	14
22 結論	14
A 常用指令	14
A.1 匯出資料集	14
A.2 訓練 policy-only 模型	14
A.3 啟動 AI 瀏覽器	15
A.4 模型互打	15

1 數據科學目標

shogiAI 的數據科學目標是讓模型從人類實戰棋譜中學會「某個局面下，下一手最可能怎麼走」。新版模型採用 policy-only 設計，也就是模型只輸出走法機率，不再輸出局面勝率或 value。

模型可表示為：

$$f_{\theta}(s) = p$$

其中 s 是目前局面， p 是長度 13,689 的 policy logits。每一個 logit 對應一種可能走法。推論時，系統只取目前局面的合法手，讀取這些合法手對應的 policy 分數，再排序給 alpha-beta 搜尋使用。

2 問題定義

本專案的機器學習任務是多類別分類。

項目	內容
輸入	將棋局面，編碼為 $43 \times 9 \times 9$ 張量。
輸出	下一手 move label，總類別數為 13,689。
標籤來源	人類實戰棋譜中的下一手。
學習方法	Supervised learning / imitation learning。
Loss	CrossEntropyLoss。
應用方式	排序合法手，輔助 alpha-beta 搜尋。

這個任務不等於直接訓練一個能獨立下棋的模型。將棋局面往往有許多合理手，實戰下一手也不一定是唯一最佳手。因此模型的主要價值是把合理候選手排到前面，讓搜尋更有效率。

3 資料來源與最新規模

資料來源包含本地 CSA 檔案、遠端爬蟲蒐集的 CSA 檔案，以及 MySQL 中已匯入的棋譜資料。最新 policy-only 訓練是在遠端大數據主機上，從 MySQL 重新匯出資料集後執行。

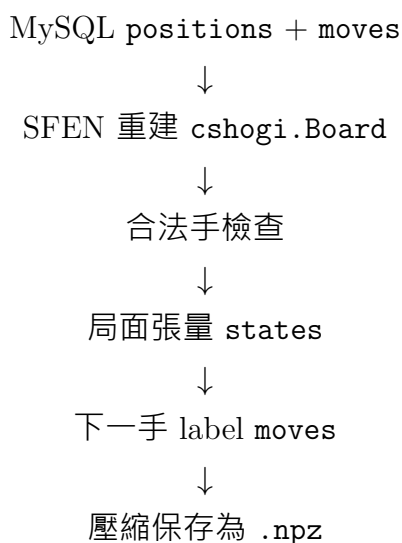
項目	最新數值
匯出資料集	out/policy_dataset.policy_only_latest.npz
對局數	42,534 局

項目	最新數值
訓練樣本總數	4,855,173 筆
非法資料列	0 筆
TORYO 樣本	4,829,692 筆
DRAW 樣本	104 筆
SENNICHITE 樣本	25,377 筆
資料集大小	約 332 MB

這些樣本是從資料庫的 `positions` 與 `moves` join 得到。每筆樣本代表「某一手前的局面」與「該局面下實戰走出的下一手」。

4 資料前處理流程

資料前處理主要由 `src/export_policy_dataset.py` 與 `src/csa_preprocess.py` 負責。整體流程如下：



核心 SQL 邏輯是將局面與下一手對齊：

```

SELECT p.game_id, p.move_number, p.sfen, m.usi_move, g.result
FROM positions p
JOIN moves m
  ON m.game_id = p.game_id
 AND m.move_number = p.move_number + 1
JOIN game_records g
  ON g.game_id = p.game_id
ORDER BY p.game_id, p.move_number;

```

在 Python 中，系統會用 `cshogi.Board(sfen)` 還原棋盤，用 `board.move_from_usi(usi_move)` 取得 `move`，再以 `board.is_legal(move)` 檢查合法性。非法資料不會進入訓練集。

5 局面特徵編碼

神經網路不能直接讀 SFEN 字串，因此系統將局面轉成固定大小張量：

$$state \in \mathbb{R}^{43 \times 9 \times 9}$$

43 個 planes 的意義如下：

Plane 範圍	意義
0–13	自方 14 種棋子在棋盤上的位置。
14–27	對方 14 種棋子在棋盤上的位置。
28–34	自方 7 種持駒數量。
35–41	對方 7 種持駒數量。
42	行棋方 plane。

這種表示法類似影像中的多通道資料。棋盤是 9×9 空間，棋種、陣營、持駒與行棋方則作為 channel。CNN 可以在這種表示中學習局部形狀，例如攻防、升變區、玉周圍防守與棋子協同。

6 行棋方視角旋轉

專案預設 `orient_to_turn=True`。若輪到後手行棋，系統會把棋盤旋轉 180 度，讓模型永遠從「目前要下棋的一方」的視角看盤。

這個設計有三個好處：

1. 減少先手與後手對稱造成的學習負擔。
2. 讓同一戰型在不同顏色下能共享特徵。
3. 讓模型輸出永遠表示「自己下一手」而不是固定先手或後手。

例如後手局面會被轉成自方在下方的視角，後手的棋子會進入「自方棋子」planes。這使 policy network 不必另外學一套後手座標系。

7 走法標籤設計

Policy head 需要固定輸出維度，但每個局面的合法手數量不同。因此系統將所有可能走法編成固定 label，再在推論時只篩選合法手。

7.1 一般移動

一般移動由來源格、目標格與是否升變組成：

$$81 \times 81 \times 2 = 13,122$$

其中 81 是棋盤格數，乘以 2 是因為同一個來源與目標可能有升變與不升變兩種。

7.2 打入持駒

將棋有持駒打入規則。可打入棋子共有 7 種，目標格有 81 格：

$$7 \times 81 = 567$$

7.3 總類別數

$$13,122 + 567 = 13,689$$

因此模型最後一層輸出 13,689 維 logits。訓練時使用實戰下一手的 label 作為 target。

8 從 Policy/Value 改成 Policy-only

專案早期曾設計 policy/value network，模型同時輸出下一手 policy 與局面 value。Value head 的概念是用最終勝負作為標籤，讓模型學習目前行棋方最後會贏、輸或和棋：

$$z \in \{-1, 0, 1\}$$

後來實作與測試後，專案改成 policy-only。主要原因如下。

8.1 Value 標籤訊號太粗

Value label 來自一盤棋最後結果，但中盤局面距離終局很遠。某一方最後獲勝，不代表他在中盤每一手都真的佔優。用最終勝負硬套到所有中間局面，會讓 value target 含有大量噪音。

8.2 投降資料使局面價值不精準

許多棋譜以 TORYO 結束。投降能指出勝負方，但無法提供每個中間局面的精確勝率。尤其在將棋中，局面可能因一手戰術而突然逆轉，用整局結果回填所有局面會讓 value head 學到不穩定訊號。

8.3 小型 CNN 容量有限

目前模型是輕量 CNN，適合快速訓練與排序候選手，但要準確評估將棋複雜局面需要更深模型、更多搜尋標籤或自我對弈。若同時要求模型學 policy 與 value，可能分散有限模型容量。

8.4 實際應用以候選手排序為主

本專案 AI 最終仍使用 alpha-beta 搜尋計算變化。神經網路最需要做的是把好手排到前面，提高剪枝效率。局面評估則由傳統評估函數、搜尋深度與 quiescence search 負責。因此移除 value head 後，系統更簡單，也更符合目前需求。

9 Policy-only 模型架構

模型定義於 `src/policy_model.py`，名稱為 `SmallPolicyNet`。架構如下：

```
Conv2d(43, 64, kernel_size=3, padding=1)
ReLU
Conv2d(64, 64, kernel_size=3, padding=1)
ReLU
Conv2d(64, 32, kernel_size=3, padding=1)
ReLU
Flatten
Linear(32 * 9 * 9, 512)
ReLU
Linear(512, 13689)
```

前半段卷積層保留棋盤空間結構，後半段 fully connected layer 將特徵轉成所有 move labels 的 logits。新版沒有 value head，因此模型只輸出 policy。

10 訓練方法

訓練腳本為 `src/train_policy.py`。新版訓練流程只包含 policy 分類。

10.1 Loss Function

使用 cross entropy：

$$\mathcal{L}_{policy} = -\log \frac{\exp(\hat{p}_a)}{\sum_j \exp(\hat{p}_j)}$$

其中 a 是實戰下一手的 move label， $\hat{p}$ 是模型輸出的 logits。

10.2 Optimizer

使用 AdamW。AdamW 將 adaptive gradient 與 weight decay 分離，通常比傳統 Adam 加 L2 regularization 更穩定。

10.3 Learning Rate Scheduler

使用 CosineAnnealingLR。Learning rate 從初始值逐漸降低到最小值，使前期快速學習，後期微調模型。

10.4 Game-level Split

資料切分不是隨機切每一手，而是依 `game_id` 切分。原因是同一盤棋相鄰局面非常相似，如果同一盤棋的前半放進訓練集、後半放進測試集，測試結果會過度樂觀。Game-level split 可以降低資料洩漏。

11 最新訓練結果

最新 policy-only 模型在遠端大數據主機訓練，並已同步回本地 `models/policy_model.pt`。模型 SHA-256 為：

```
79bee4f2299667004803ec2265ba67a06c3236d08735372a7594a711b34b0ab0
```

項目	數值
Train samples	3,882,230
Validation samples	487,292
Test samples	485,651
Test policy loss	2.2751
Top-1 accuracy	41.10%
Top-3 accuracy	65.24%
Top-5 accuracy	74.49%

初始局面的前五個 policy 候選手如下：

走法	機率
7g7f	61.10%
2g2f	33.92%
5g5f	3.34%
2h7h	1.15%

走法	機率
9g9f	0.15%

12 結果解讀

Top-1 accuracy 表示模型直接猜中實戰下一手的比例。最新模型 Top-1 為 41.10%，代表在測試局面中，模型第一候選手有約四成與實戰相同。

Top-5 accuracy 更適合評估本專案用途。將棋每個局面常有多個合理手，而模型主要用於搜尋前的 move ordering。Top-5 為 74.49%，表示實戰下一手大約四分之三會出現在模型前五候選手中，這對 alpha-beta 剪枝很有幫助。

13 模型推論與快取

推論封裝為 PolicyPredictor。它負責：

- 載入 .pt checkpoint。
- 將 cshogi.Board 轉成 $43 \times 9 \times 9$ state。
- 執行模型取得 logits。
- 只取合法手對應的 label 分數。
- 對合法手分數做 softmax，得到候選手機率。
- 依 score 排序回傳 PolicyCandidate。
- 用 board.zobrist_hash() 做 prediction cache，減少重複局面推論。

這裡的 softmax 只在合法手集合內計算，因此機率表示「在目前合法手中，模型偏好此手的比例」。

14 Alpha-beta 搜尋概論

只靠 policy network 直接選最高機率手，容易漏看短期戰術，例如吃子交換、王手、詰將棋或防守手。因此 shogiAI 將神經網路與傳統搜尋結合。搜尋核心位於 src/ai_search.py。

14.1 Minimax 與 Negamax

棋類遊戲是雙人零和遊戲。若一方得分越高，另一方就越低。Minimax 的想法是：我方選擇能讓自己最好的走法，對方也會選擇讓我最差的應手。

Negamax 是 minimax 的簡化形式，利用零和性質：

$$score(s) = \max_a -score(T(s, a))$$

其中 $T(s, a)$ 表示在局面 s 下走 a 後的新局面。每往下一層，視角會切換到對手，所以分數取負。

14.2 Alpha-beta Pruning

Alpha-beta 是 minimax 的剪枝最佳化。它維護兩個界線：

- α ：目前已知我方至少能拿到的分數。
- β ：目前對方允許我方拿到的上限。

若搜尋中發現某分支已不可能影響最終選擇，就停止展開該分支。剪枝不改變理論結果，但可大幅減少節點數。

15 shogiAI 搜尋技術

15.1 Quiescence Search

普通 alpha-beta 若剛好在戰術交換中停止，可能出現 horizon effect。例如葉節點看起來我方多一子，但下一手對方能吃回。Quiescence search 會在葉節點繼續延伸戰術手，例如吃子、升變或王手，直到局面較安靜再評估。

15.2 Transposition Table

不同走法順序可能到達同一局面。Transposition table 用 Zobrist hash 記錄已搜尋局面的分數、深度與最佳手。若搜尋再次遇到相同局面，可以直接使用快取結果或縮小 alpha-beta 範圍。

15.3 Killer Move

Killer move heuristic 記錄在同一搜尋深度曾造成 beta cutoff 的安靜手。若某手曾讓對手無法接受，之後同層搜尋可優先嘗試它，提高剪枝機會。

15.4 History Heuristic

History heuristic 統計某個走法在過去搜尋中造成 cutoff 的頻率與深度。越常在深層造成 cutoff 的走法，排序分數越高。

15.5 Late Move Reduction

若走法排序已經把較可能的好手放前面，後面排序很低且不是王手、不是戰術手的走法，可以先用較低深度搜尋。若低深度結果意外很好，再回頭完整搜尋。這可以節省大量時間。

16 手工評估函數

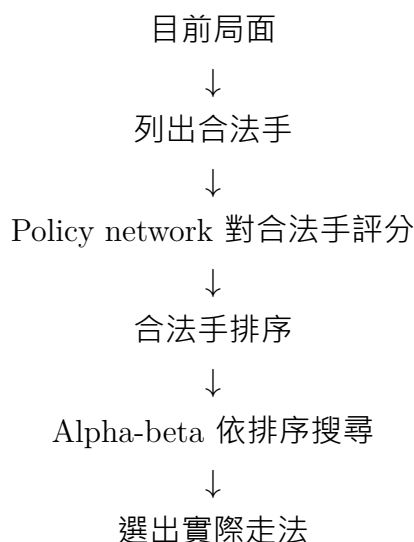
搜尋到葉節點時，需要評估局面分數。evaluate_position 不是只算子力，還加入多種將棋特徵：

- **子力價值**：飛、角、金、銀、桂、香、步與升變子的基礎價值。
- **持駒價值**：將棋持駒可打入，因此持駒也有分數。
- **位置分數**：棋子靠近中央、進入升變區或位置活躍會加分。
- **機動力**：可行走法越多，局面越靈活。
- **玉安全**：玉周圍保護、敵方壓力與直線攻擊。
- **懸空棋子**：被攻擊但保護不足的棋子會扣分。
- **持駒壓力**：持駒對敵玉周圍可打入威脅會加分。
- **節奏**：輪到行棋方可能有主動權 bonus。

這些手工特徵提供基本棋力，使 AI 即使沒有神經網路，也能以 alpha-beta 下出合理走法。

17 Policy Network 如何結合 Alpha-beta

Policy network 不直接取代 alpha-beta，而是作為 move ordering。



Alpha-beta 的效率非常依賴走法排序。如果好手越早被搜尋， α 或 β 越快更新，就能越早剪掉其他分支。新版模型 Top-5 accuracy 達 74.49%，表示它能有效把實戰合理手排在前面。

18 Opening Book

除了 policy network，AI 開局也可使用 opening book。Opening book 由 MySQL 統計前若干手常見局面與下一手頻率。若目前 SFEN 在 book 中，且某走法出現次數達到門檻，AI 可以直接採用該走法。

Opening book 的優點是：

- 開局階段快速，不必消耗搜尋時間。
- 走法來自真實棋譜，較符合人類常見戰型。
- 可避免早期搜尋深度不足導致怪異開局。

19 模型互打與評估

`src/compare_ai_models.py` 可讓兩個模型互相對弈。它會交替先後手，避免先手優勢影響判斷。輸出包含：

- 新模型勝場。
- 舊模型勝場。
- 和棋數。
- 新模型得分率。

- 平均手數。
- 每盤棋 USI 手順。

需要注意的是，Top-k accuracy 衡量的是「模仿實戰下一手」，模型互打才更接近棋力評估。但模型互打需要較多局數、固定深度與固定時間限制，結果才有統計意義。

20 前端展示與應用

前端 AI 對弈頁面會顯示：

- 搜尋評分。
- 搜尋深度。
- 搜尋節點數。
- PV，也就是主要變化。
- Policy 候選手與機率。

新版前端已移除「局面值」欄位，因為模型不再輸出 value。這能避免使用者誤以為神經網路仍在估計勝率。

21 限制與改進方向

21.1 模型容量限制

目前模型是小型 CNN，訓練與推論速度快，但容量有限。若要提升 policy 準確率，可改用 residual CNN 或 attention-based 架構。

21.2 資料標籤限制

模型學的是人類實戰下一手，而不是數學上的最佳手。同一局面可能有多個好手，但資料只標一手。因此 Top-1 不應被解讀為唯一棋力指標。

21.3 Legal Move Mask

目前模型輸出所有 13,689 類，推論時再挑合法手。未來可在 loss 中加入合法手 mask 或改良 label 設計，讓模型更專注於實際可行走法。

21.4 自我對弈

若要讓模型不只模仿人類，可加入 self-play。透過目前 AI 搜尋產生新棋譜，再用結果訓練模型，逐步提升棋力。

21.5 Value 的未來可能

Value head 目前被移除，不代表永遠不能使用。若未來要重新嘗試 value，建議不要直接用最終勝負套到所有中間局面，而是使用搜尋評分、自我對弈結果、或更細緻的 bootstrap target。這樣 value 訊號會比單純終局勝負更穩定。

22 結論

shogiAI 的數據科學流程已從早期 policy/value 模型調整為 policy-only 模型。新版設計讓神經網路專注於從大量實戰棋譜中學習下一手分布，並把學到的知識用於 alpha-beta 搜尋的走法排序。最新資料集包含 4,855,173 筆樣本，測試 Top-1 accuracy 達 41.10%，Top-5 accuracy 達 74.49%。整體系統結合了資料庫、特徵工程、CNN 分類模型、opening book 與傳統棋類搜尋，使專案具備從資料蒐集到實際 AI 對弈的完整流程。

A 常用指令

A.1 匯出資料集

```
python src\export_policy_dataset.py ^
--output out\policy_dataset.policy_only_latest.npz ^
--host 140.135.65.53 ^
--port 3306 ^
--user 11211213 ^
--database DB11211213
```

A.2 訓練 policy-only 模型

```
python src\train_policy.py ^
--input out\policy_dataset.policy_only_latest.npz ^
--output out\policy_model.policy_only.pt ^
--batch-size 256
```

A.3 啟動 AI 瀏覽器

```
python src\csa_browser_api.py ^  
  --source mysql ^  
  --db-host 140.135.65.53 ^  
  --db-port 3306 ^  
  --db-user 11211213 ^  
  --db-name DB11211213 ^  
  --policy-model models\policy_model.pt ^  
  --policy-order-ply 2
```

A.4 模型互打

```
python src\compare_ai_models.py ^  
  --old-model out\policy_model.prev.pt ^  
  --new-model models\policy_model.pt ^  
  --games 20 ^  
  --depth 3 ^  
  --time-limit-ms 1000
```